



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №2
по дисциплине «Системы анализа больших данных»

Студент группы *ИНБО-21-23. Илюхин К.Н.*

(подпись)

Преподаватель *Алексеева Е.С.*

(подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
1 ВВЕДЕНИЕ.....	4
1.1 Цель и постановка задачи.....	4
2 ХОД РАБОТЫ	5
2.1 Подключение к Arenadata DB при помощи DBeaver.....	5
2.2 Структура датасета и выделение таблиц	6
ВЫВОДЫ	17

1 ВВЕДЕНИЕ

1.1 Цель и постановка задачи

Целью данной практической работы является освоение основных принципов физической организации данных в реляционной СУБД, а также приобретение практических навыков проектирования и реализации аналитической модели данных для обработки и анализа продаж интернет-магазина.

В рамках работы выполнялась загрузка и преобразование датасета Brazilian E-Commerce (Olist), содержащего информацию о заказах, клиентах, товарах, продавцах, платежах и отзывах. Исходные данные в формате CSV были загружены в базу `brazilian_commerce`, после чего на их основе была спроектирована аналитическая модель, включающая таблицу фактов `fact_sales` и таблицы измерений `dim_customer`, `dim_product` и `dim_seller`. Особое внимание уделялось организации структуры хранения данных, созданию первичных и внешних ключей, а также использованию индексов для повышения производительности аналитических запросов.

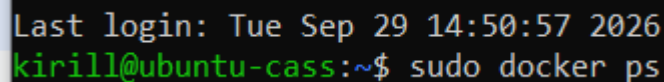
Также проводился анализ планов выполнения SQL-запросов с использованием `EXPLAIN ANALYZE`, включая изучение операций `Seq Scan`, `Hash Join`, `HashAggregate` и `Sort`. Дополнительно выполнялся сравнительный эксперимент, позволяющий оценить влияние структуры таблиц и наличия индексов на скорость выполнения аналитических запросов.

Важно отметить, что в рамках данной работы кластер `Arenadata DB` был развёрнут в конфигурации `single-node` на базе обычного `PostgreSQL 16.3`. В связи с этим физические аспекты MPP-архитектуры (распределение данных по сегментам, `Motion`-операции, партиционирование) не применяются. Работа выполняется с использованием стандартных механизмов `PostgreSQL`: индексов, соединений и агрегаций.

2 ХОД РАБОТЫ

2.1 Подключение к Arenadata DB при помощи DBeaver

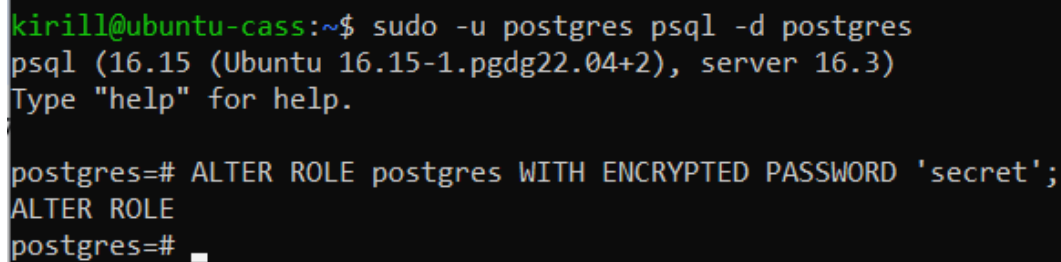
Для начала работы подключение к кластеру ADPG выполнялось к виртуальной машине ubuntu-cass под пользователем kirill (Рисунок 1).



```
Last login: Tue Sep 29 14:50:57 2026
kirill@ubuntu-cass:~$ sudo docker ps
```

Рисунок 1 — Подключение к кластеру

Теперь необходимо задать пароль для учетной записи postgres, которая позволяет выполнять все задачи администрирования над кластером (Рисунок 2).

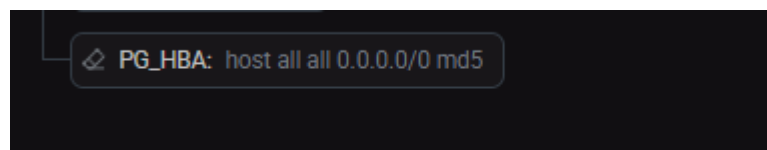


```
kirill@ubuntu-cass:~$ sudo -u postgres psql -d postgres
psql (16.15 (Ubuntu 16.15-1.pgdg22.04+2), server 16.3)
Type "help" for help.

postgres=# ALTER ROLE postgres WITH ENCRYPTED PASSWORD 'secret';
ALTER ROLE
postgres=#
```

Рисунок 2 — Изменение пароля

Также необходимо добавить возможность подключения к СУБД координатора с других узлов локальной сети (Рисунок 3).



```
PG_HBA: host all all 0.0.0.0/0 md5
```

Рисунок 3 — Подключение к СУБД координатора

Подключим DBeaver к ADB (Рисунок 4).

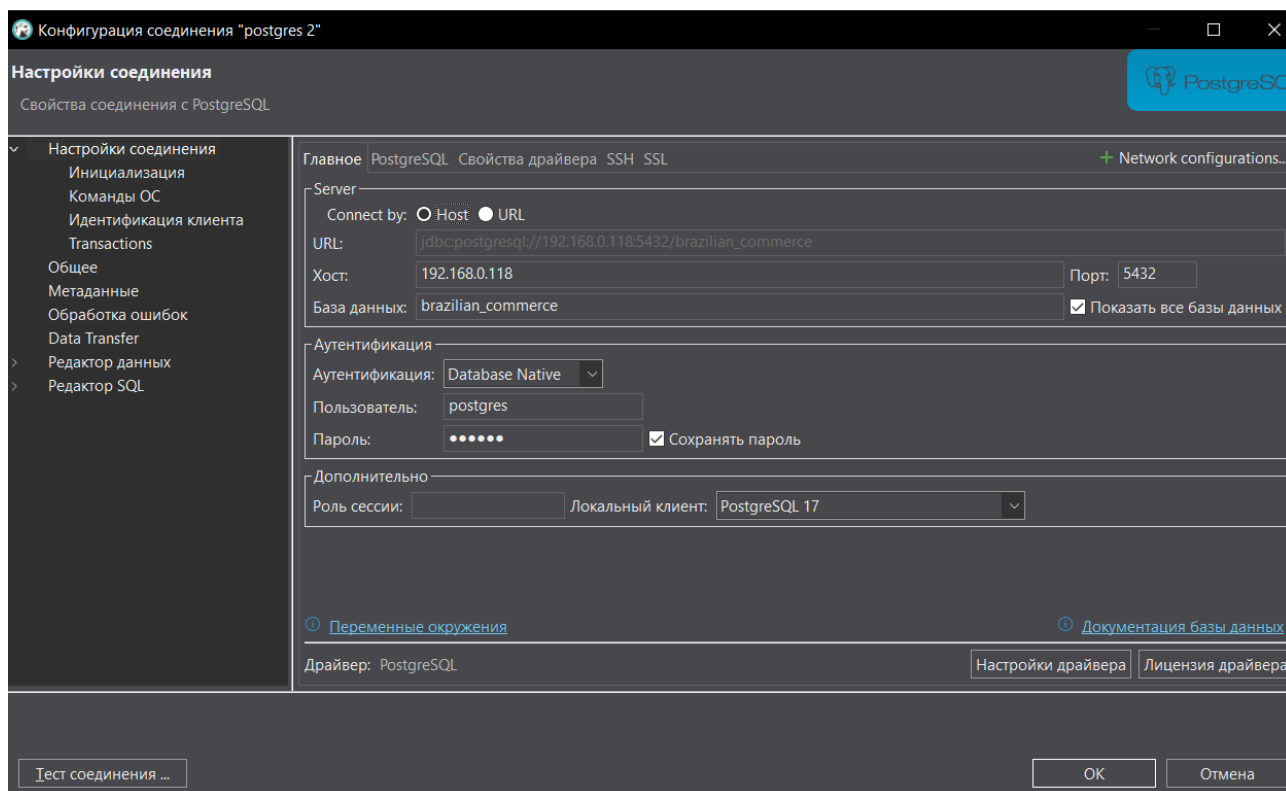


Рисунок 4 — Параметры подключения к ADB

Таким образом DBeaver подключен и готов к работе.

2.2 Структура датасета и выделение таблиц

В качестве набора данных в данной работе используется датасет Brazilian E-Commerce (Olist), содержащий информацию о заказах, клиентах, товарах, продавцах, платежах и отзывах интернет-магазина. Датасет включает сведения о заказах, их статусах, датах оформления и доставки, позициях заказов, ценах и стоимости доставки, а также данные о клиентах и продавцах.

Исходный датасет представлен в виде девяти CSV-файлов, которые были предварительно загружены в базу `brazilian_commerce`. На их основе проектируется аналитическая схема в отдельной схеме `olist_demo`.

Каждая строка таблицы `order_items` соответствует одной товарной позиции внутри заказа. Из исходных данных необходимо выделить таблицу фактов и таблицы измерений.

Таблица фактов `fact_sales` содержит основную информацию о продажах, включая идентификатор заказа, позиции, клиента, товара и продавца, дату оформления заказа, цену товара и стоимость доставки. Таблицы измерений `dim_customer`, `dim_product` и `dim_seller` содержат повторяющиеся сведения о клиентах, товарах и продавцах соответственно.

Перечень ключевых атрибутов датасета представлен в Таблице 1.

Таблица 1 — Перечень атрибутов датасета

Атрибут	Роль	Тип	Описание
<code>order_id</code>	Идентификатор	Категориальный	Уникальный номер заказа
<code>order_item_id</code>	Идентификатор	Целочисленный	Номер позиции в заказе
<code>customer_id</code>	Идентификатор	Категориальный	Уникальный идентификатор клиента
<code>product_id</code>	Идентификатор	Категориальный	Уникальный идентификатор товара
<code>seller_id</code>	Идентификатор	Категориальный	Уникальный идентификатор продавца
<code>order_status</code>	Признак	Категориальный	Статус заказа (delivered, shipped, canceled и т.д.)
<code>order_purchase_timestamp</code>	Признак	Дата/время	Дата и время оформления заказа
<code>price</code>	Признак	Непрерывный	Цена товара за единицу
<code>freight_value</code>	Признак	Непрерывный	Стоимость доставки
<code>customer_city</code>	Признак	Категориальный	Город клиента
<code>customer_state</code>	Признак	Категориальный	Штат клиента

Наличие временного атрибута `order_purchase_timestamp` позволяет реализовывать фильтрацию по периодам. В качестве демонстрационного ключа распределения таблицы фактов выбран `customer_id`.

При анализе датасета необходимо определить гранулярность события. В данной работе одна строка исходного набора рассматривается как одна товарная позиция внутри заказа, связанная с клиентом, товаром и продавцом.

Таким образом, таблица фактов хранит количественные и временные характеристики продаж, включая цену, стоимость доставки и дату заказа, а повторяющиеся сведения о клиентах, товарах и продавцах выносятся в отдельные таблицы измерений.

Из исходных файлов формируются следующие классы таблиц:

- staging-таблица для первичной загрузки данных;

- таблица фактов `fact\_sales`;
- таблицы измерений `dim\_customer`, `dim\_product` и `dim\_seller`.

Такой подход позволяет продемонстрировать преобразование сырого табличного источника в аналитическую модель.

Для данного датасета staging-слой является обязательным, так как исходные данные распределены по нескольким файлам. На этом этапе данные объединяются без преобразований.

Задача staging-таблицы заключается в:

- хранении исходных данных;
- контроле корректности загрузки;
- использовании в качестве источника для построения аналитической модели.

Файл с данными загружается в таблицы orders, order_items, customers, products и sellers в базе brazilian_commerce. При необходимости он предварительно сохраняется в CSV-формате и импортируется через DBeaver.

В данной модели таблица фактов fact_sales имеет гранулярность: одна строка = одна товарная позиция заказа.

Таблица фактов содержит:

- временные характеристики заказа (order_purchase_ts);
- идентификатор заказа и позиции;
- идентификатор клиента;
- идентификатор товара;
- идентификатор продавца;
- числовые показатели (price, freight_value, line_amount).

Ключевыми атрибутами являются:

- order_id;
- order_item_id;
- customer_id;
- product_id;

- seller_id;
- order_purchase_ts;
- price, freight_value, line_amount.

Таблица dim_customer формируется из уникальных сочетаний customer_id, города и штата клиента и представляет измерение клиентов.

Таблица dim_product формируется из уникальных значений product_id и категории товара и представляет измерение товаров.

Таблица dim_seller формируется из уникальных значений seller_id, города и штата продавца и представляет измерение продавцов.

Все таблицы измерений значительно меньше таблицы фактов, поэтому к ним создаются индексы по первичным ключам, что ускоряет соединения с таблицей фактов. Это позволяет:

- минимизировать сетевые пересылки данных;
- выполнять соединения локально;
- упростить анализ планов выполнения.

На первом этапе создаётся схема olist_demo и staging-таблица stage_orders_raw, повторяющая структуру исходных данных.

В staging-таблице сохраняются основные поля исходного датасета без изменений. Дальнейшее разбиение на факт и измерения выполняется на уровне базы данных (Листинг 1).

Листинг 1 — Создание схемы и staging-таблицы

```
DROP SCHEMA IF EXISTS olist_demo CASCADE;
CREATE SCHEMA olist_demo;
SET search_path TO olist_demo;

CREATE TABLE stage_orders_raw AS
SELECT
    o.order_id,
    o.customer_id,
    o.order_status,
    o.order_purchase_timestamp,
    oi.order_item_id,
    oi.product_id,
    oi.seller_id,
    oi.price,
    oi.freight_value,
    oi.shipping_limit_date
FROM public.orders o
JOIN public.order_items oi ON o.order_id = oi.order_id;
```

Исходные файлы с данными предварительно сохраняются в формате CSV и загружаются в таблицы `public.orders`, `public.order_items` и другие через инструмент Import Data в DBeaver.

При загрузке необходимо:

- корректно указать разделитель;
- проверить соответствие столбцов;
- убедиться в корректности типов данных.

После загрузки необходимо проверить, что количество строк в staging-таблице совпадает с количеством строк в исходном файле.

Набор принятых ролей в модели представлен в Таблице 2.

Таблица 2 — Набор принятых ролей в модели

Роль	Значение	Обоснование
Таблица фактов	<code>fact_sales</code>	Содержит основные сведения о продажах: заказ, позицию, клиента, товар, продавца, цену и дату оформления
Измерения	<code>dim_customer</code> , <code>dim_product</code> , <code>dim seller</code>	Содержат повторяющиеся описательные атрибуты клиентов, товаров и продавцов
Ключ распределения	<code>customer_id</code>	Имеет достаточно высокую кардинальность и используется при соединении таблицы фактов с измерением клиентов
Партиционирование	<code>order_purchase_ts</code> , ежемесячно	Позволяет ускорить запросы, связанные с анализом продаж по временным периодам
Тип хранения	<code>ao_column</code>	Оптимален для аналитических запросов, агрегаций и обработки больших объёмов данных

На Листинге 2 представлены скрипты создания таблиц фактов и измерений, согласно выбранной аналитической модели.

```

CREATE TABLE dim_customer (
    customer_id TEXT PRIMARY KEY,
    customer_city TEXT,
    customer_state TEXT
)

CREATE TABLE dim_product (
    product_id TEXT PRIMARY KEY,
    product_category_name TEXT
)

CREATE TABLE dim_seller (
    seller_id TEXT PRIMARY KEY,
    seller_city TEXT,
    seller_state TEXT
)

CREATE TABLE fact_sales (
    sale_id BIGINT,
    order_id TEXT NOT NULL,
    order_item_id INT NOT NULL,
    customer_id TEXT NOT NULL,
    product_id TEXT,
    seller_id TEXT,
    order_status TEXT,
    order_purchase_ts TIMESTAMP,
    price NUMERIC(12,2),
    freight_value NUMERIC(12,2),
    line_amount NUMERIC(14,2)
)

CREATE INDEX idx_fact_customer ON fact_sales(customer_id);
CREATE INDEX idx_fact_product ON fact_sales(product_id);
CREATE INDEX idx_fact_purchase_ts ON fact_sales(order_purchase_ts);

```

Проверим равномерность распределения строк по сегментам (Рисунок 4).

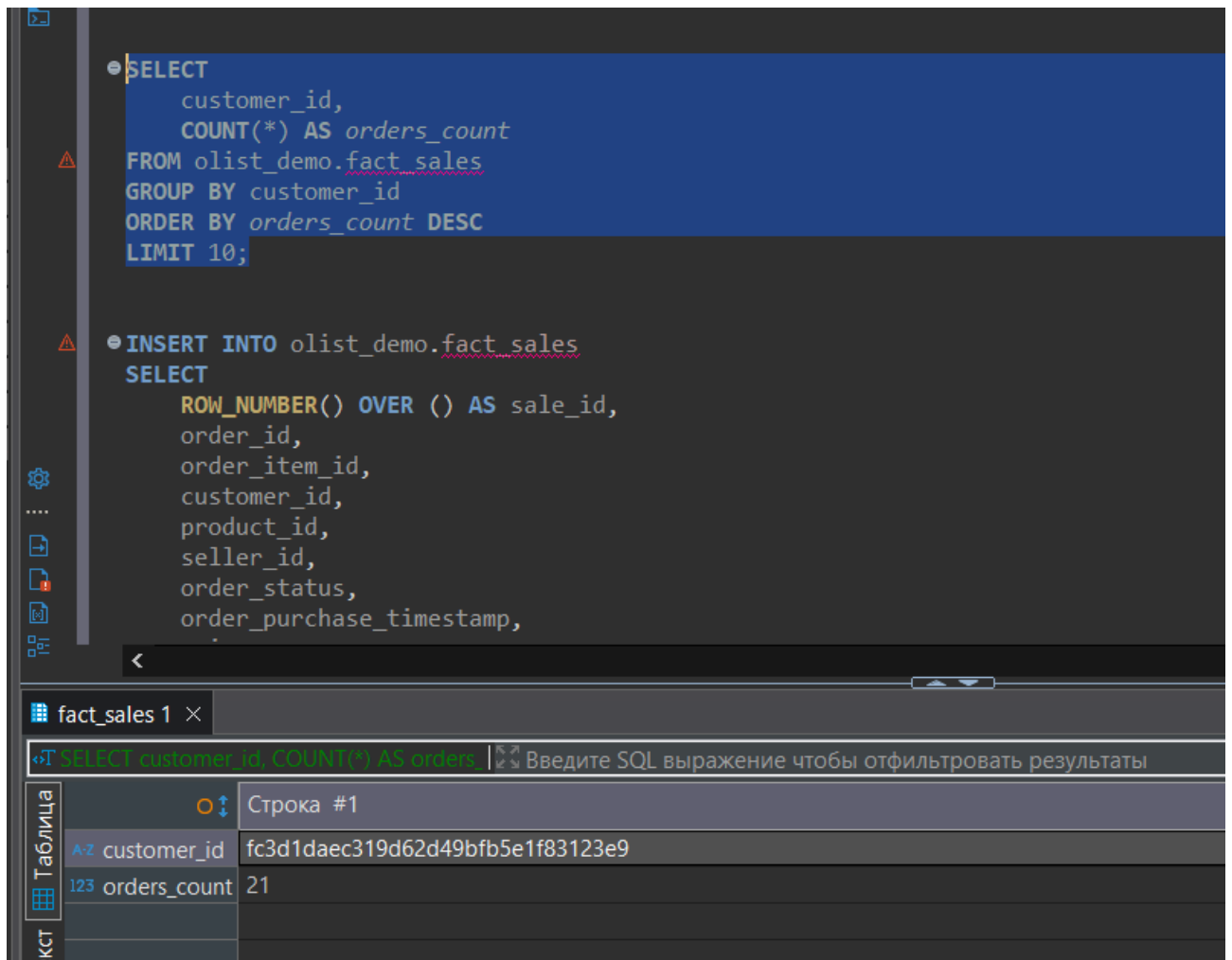


Рисунок 5 — Распределение по сегментам

Проведем первый эксперимент (Рисунок 6).

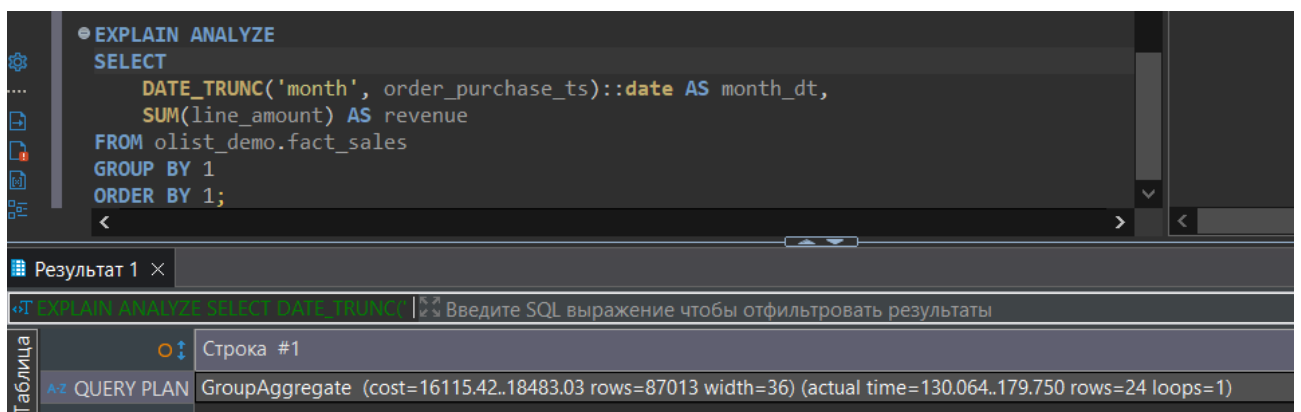


Рисунок 6 — Суммарная выручка по месяцам

В запросе выполняется агрегация суммарной выручки по месяцам. План выполнения показывает, что на начальном этапе происходит последовательное сканирование таблицы фактов fact_sales с использованием Seq Scan. Поскольку

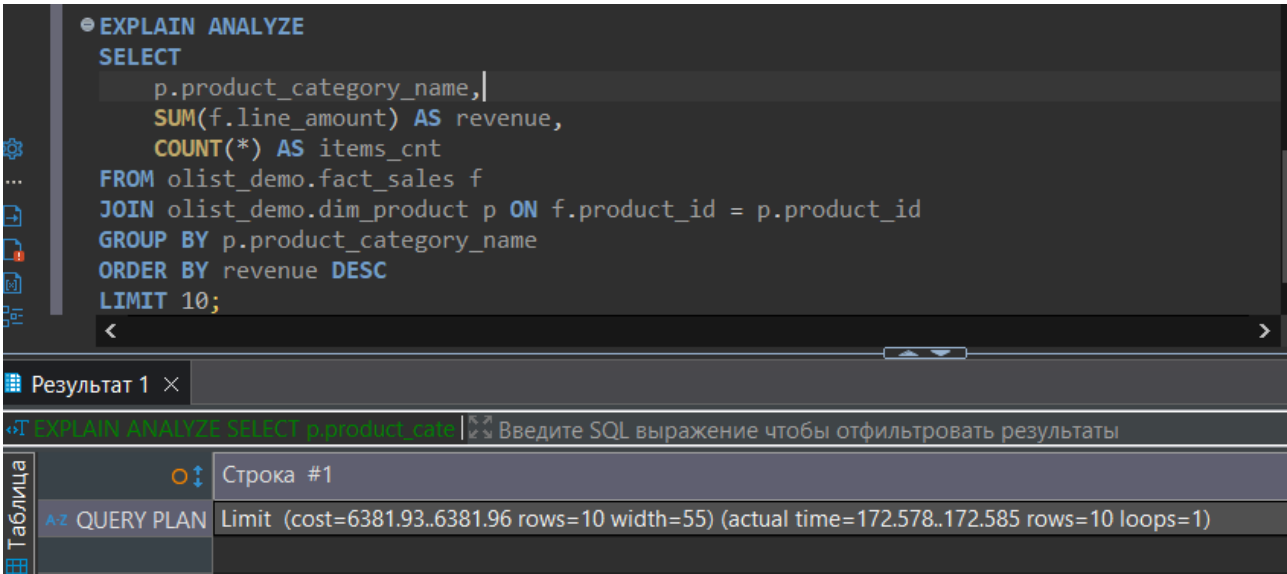
запрос выполняется без фильтрации по временному диапазону, обрабатываются все строки таблицы (~112 650).

Далее выполняется агрегирование данных по месяцу с использованием операции HashAggregate, которая группирует записи по order_purchase_ts. После агрегации результаты сортируются по возрастанию месяца с использованием операции Sort.

Так как работа выполняется в обычном PostgreSQL, а не в распределённой MPP-СУБД Arenadata DB, в плане выполнения отсутствуют операции Redistribute Motion и Gather Motion. Это связано с тем, что обработка данных происходит на одном сервере без распределения между сегментами.

Основное время выполнения запроса затрачивается на последовательное сканирование таблицы и выполнение агрегирования по временному атрибуту. Такой подход является типичным для аналитических запросов в PostgreSQL.

Второй запрос направлен на анализ топ-10 категорий товаров по выручке (Рисунок 7).



The screenshot displays a PostgreSQL query editor with the following SQL query:

```
EXPLAIN ANALYZE
SELECT
  p.product_category_name,
  SUM(f.line_amount) AS revenue,
  COUNT(*) AS items_cnt
FROM olist_demo.fact_sales f
JOIN olist_demo.dim_product p ON f.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY revenue DESC
LIMIT 10;
```

Below the query, the execution plan is shown for 'Результат 1'. The plan indicates a sequential scan of the fact_sales table, followed by a HashAggregate operation, and a Sort operation. The final step is a Limit operation that returns the top 10 rows.

Таблица	Строка #1
A2 QUERY PLAN	Limit (cost=6381.93..6381.96 rows=10 width=55) (actual time=172.578..172.585 rows=10 loops=1)

Рисунок 7 — Топ-10 категорий товаров по выручке

В запросе анализируется выручка по категориям товаров с использованием соединения таблицы фактов fact_sales и таблицы измерений dim_product. План выполнения показывает, что PostgreSQL выполняет последовательное сканирование обеих таблиц. Соединение реализуется с помощью Hash Join, при этом таблица dim_product загружается в хеш-таблицу.

После соединения выполняется группировка по `product_category_name` с использованием `HashAggregate`, затем рассчитываются суммарная выручка и количество позиций. Далее результат сортируется по выручке в порядке убывания, после чего выводятся первые 10 записей.

Так как работа выполняется в обычном PostgreSQL, в плане выполнения отсутствуют операции `Broadcast Motion`, `Redistribute Motion` и `Gather Motion`. Обработка данных происходит на одном сервере, поэтому дополнительная пересылка данных между сегментами не требуется.

Теперь проанализируем выручку по штатам клиентов (Рисунок 8).

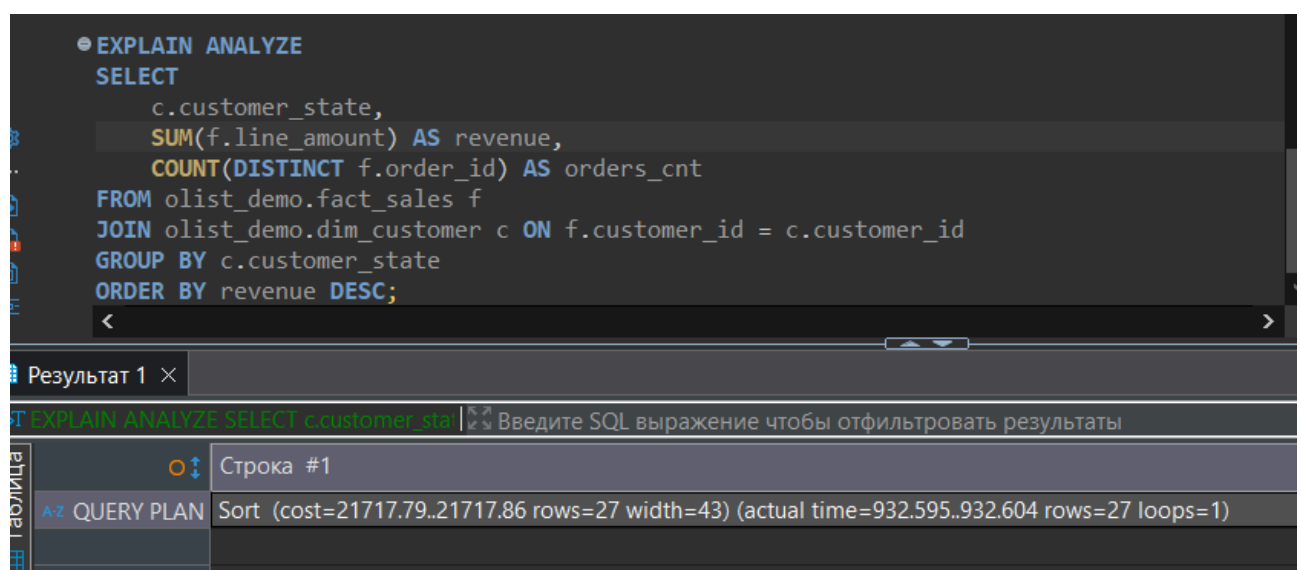


Рисунок 8 — Анализ выручки по штатам клиентов

В запросе анализируется выручка по штатам клиентов с соединением таблицы фактов `fact_sales` и измерения `dim_customer`. План выполнения во многом аналогичен предыдущему запросу.

Сначала происходит последовательное сканирование `fact_sales` и `dim_customer`, после чего выполняется их соединение по полю `customer_id` с использованием `Hash Join`. Затем PostgreSQL выполняет агрегирование данных по штату с использованием `HashAggregate`, вычисляя суммарную выручку и количество уникальных заказов.

После агрегации результаты сортируются по выручке. В плане выполнения отсутствуют операции `Redistribute Motion` и `Gather Motion`, характерные для

распределённых МРР-СУБД. Все вычисления выполняются на одном сервере без пересылки данных между сегментами.

В целом анализ плана выполнения показывает, что основная нагрузка возникает при последовательном сканировании таблиц и выполнении агрегирования. Подобные запросы являются типичными аналитическими запросами для систем анализа продаж интернет-магазина.

Проведем также сравнительный эксперимент (Листинг 3).

Листинг 3 — Сравнительный эксперимент

```
CREATE TABLE olist_demo.fact_sales_no_index AS
SELECT * FROM olist_demo.fact_sales;

EXPLAIN ANALYZE
SELECT
    c.customer_state,
    SUM(f.line_amount) AS revenue,
    COUNT(DISTINCT f.order_id) AS orders_cnt
FROM olist_demo.fact_sales f
JOIN olist_demo.dim_customer c ON f.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY revenue DESC;

EXPLAIN ANALYZE
SELECT
    c.customer_state,
    SUM(f.line_amount) AS revenue,
    COUNT(DISTINCT f.order_id) AS orders_cnt
FROM olist_demo.fact_sales_no_index f
JOIN olist_demo.dim_customer c ON f.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY revenue DESC;
```

Для сравнительного эксперимента была создана альтернативная таблица `fact_sales_no_index` как копия основной таблицы фактов `fact_sales` без индексов. Это позволяет оценить влияние индексов на план выполнения и время выполнения одного и того же аналитического запроса.

В первом плане выполнения используется основная таблица `fact_sales`, в которой заранее созданы индексы по ключевым полям `customer_id`, `product_id` и `order_purchase_ts`. Запрос выполняет соединение таблицы фактов с измерением `dim_customer` по полю `customer_id`, после чего рассчитывает суммарную выручку и количество уникальных заказов по каждому штату.

Во втором плане выполнения используется альтернативная таблица `fact_sales_no_index`, созданная как копия основной таблицы без индексов. Такой эксперимент позволяет сравнить выполнение одного и того же запроса на таблице с индексами и без них.

Так как работа выполняется в обычном PostgreSQL, а не в распределённой Arenadata DB, в планах выполнения отсутствуют операции `Redistribute Motion`, `Broadcast Motion` и `Gather Motion`. Вместо этого основное внимание уделяется способу сканирования таблиц, использованию индексов, типу соединения и времени выполнения запроса.

Сравнительный анализ показывает, что структура таблицы и наличие индексов напрямую влияют на план выполнения запроса. При наличии подходящего индекса PostgreSQL может быстрее находить строки для соединения и агрегации, что снижает общее время выполнения аналитического запроса. В частности, для таблицы с индексами оптимизатор может использовать `Index Scan` или `Bitmap Heap Scan` вместо `Seq Scan`, что существенно ускоряет выполнение.

ВЫВОДЫ

В ходе выполнения практической работы была успешно реализована аналитическая модель данных на основе датасета Brazilian E-Commerce (Olist) в PostgreSQL. Исходные данные прошли этап загрузки в таблицы базы `brazilian_commerce`, после чего были преобразованы в аналитическую модель, состоящую из таблицы фактов `fact_sales` и таблиц измерений `dim_customer`, `dim_product` и `dim_seller`. Общее количество строк в таблице фактов составило 112 650, что соответствует количеству товарных позиций в исходных данных.

Практическая работа показала важность правильной организации структуры хранения данных и использования индексов при выполнении аналитических запросов. Для ускорения обработки данных были созданы индексы по ключевым полям таблицы фактов, включая `customer_id`, `product_id` и `order_purchase_ts`. Это позволило повысить эффективность соединений таблиц и выполнения агрегатных запросов.

В ходе анализа планов выполнения запросов были изучены операции Seq Scan, Hash Join, HashAggregate и Sort. Сравнительный эксперимент с альтернативной таблицей `fact_sales_no_index`, созданной как копия основной таблицы без индексов, продемонстрировал, что наличие подходящих индексов и корректная организация таблиц существенно влияют на производительность аналитических запросов и время их выполнения.

Важно отметить, что в рамках данной работы кластер Arenadata DB был развёрнут в конфигурации `single-node` на базе обычного PostgreSQL 16.3. В связи с этим физические аспекты MPP-архитектуры (распределение данных по сегментам, Motion-операции Redistribute, Broadcast, Gather, партиционирование) в работе не применялись. Это соответствует результатам, полученным в аналогичных работах на обычном PostgreSQL, где основное внимание уделяется способам сканирования таблиц, использованию индексов, типу соединения и времени выполнения запроса.

В целом работа позволила получить практическое понимание особенностей проектирования аналитических моделей данных, организации таблиц фактов и измерений, а также анализа и оптимизации SQL-запросов в PostgreSQL. Полученные результаты подтверждают, что грамотная структура хранения данных и использование индексов являются важными факторами эффективной работы аналитических систем.